Основные понятия

- **Криптология** – наука, которая делится на две науки: **криптография** и **криптоанализ**.

- **Криптография (алгоритмы шифрования)** – наука, которая изучает преобразования, делающие смысл информации непригодным для злоумышленника.

- **Криптоанализ (взлом алгоритмов)** – наука, которая изучает нахождение смысла информации без доступа к секретному параметру системы (ключу).

- **Ключ** (секретный параметр системы шифрования) – уникальное состояние некоторого параметра системы, которое позволяет из множества возможных отображений выбрать нужное.

- **Шифр** – математическая функция, которая преобразовывает открытый блок данных на основе секретного параметра (ключа).

То есть **шифр** – это алгоритм, с помощью которого выполняется и **шифрование** и **расшифрование** данных.

- **Шифрование** – процесс нормального применения криптографического преобразования открытого текста на основе алгоритма и ключа, в результате которого возникает зашифрованный текст.

- **Расшифровывание** – процесс нормального применения криптографического преобразования шифрованного текста в открытый.

- **Дешифрование (взлом, дешифровка)** – процесс извлечения открытого текста без знания криптографического ключа на основе известного шифрованного.

Термин «дешифрование» обычно применяют по отношению к процессу криптоанализа шифротекста.

Однако криптоанализ может заключаться и в анализе самой шифросистемы, а не только зашифрованного ею открытого сообщения.

- **Криптографическая стойкость** — способность криптографического алгоритма противостоять криптоанализу.

Основные признаки любого шифра

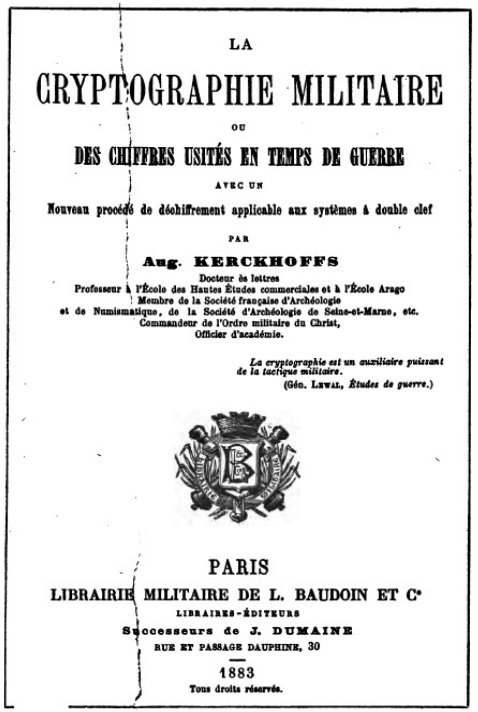
- Имеет ключ
- Подчиняется принципам Керкгоффса
- Шифрование != кодирование



Огюст Керкгоффс (1835 - 1903)

Голландский криптограф, лингвист, историк, математик

Автор фундаментального труда «Военная криптография» («La Cryptographie Militaire»)



LA

CRYPTOGRAPHIE MILITAIRE

OU

DES CHIFFRES USITÉS EN TEMPS DE GUERRE

AVEC UN

Nouveau procédé de déchiffrement applicable aux systèmes à double clef

PAR

Aug. KERCKHOFFS

Docteur ès lettres
Professeur à l'École des Hautes Études commerciales et à l'École Arago
Membre de la Société française d'Archéologie
et de Numismatique, de la Société d'Archéologie de Seine-et-Marne, etc.
Commandeur de l'Ordre militaire du Christ,
Officier d'académie.

La cryptographie est un auxiliaire puissant de la tactique militaire.
(Gén. Lewal, *Études de guerre.*)

PARIS

LIBRAIRIE MILITAIRE DE L. BAUDOIN ET Cie

LIBRAIRES-ÉDITEURS

Successeurs de J. DUMAINE

RUE ET PASSAGE DAUPHINE, 30

1883

Tous droits réservés.

Сформулировал общие требования к криптосистеме, а также утвердил криптоанализ как единственно верный способ испытания шифров

Принципы Керкгоффса (1883 г.)

1. Система должна быть физически, если не математически, невскрываемой.
2. Нужно, чтобы не требовалось сохранение системы в тайне; попадание системы в руки врага не должно причинять неудобств.
3. Хранение и передача ключа должны быть осуществимы без помощи бумажных записей; корреспонденты должны располагать возможностью менять ключ по своему усмотрению.

Принципы Керкгоффса (1883 г.)

4. Система должна быть пригодной для сообщения через телеграф (в настоящее время – по сети).
5. Система должна быть легко переносимой (портативной), работа с ней не должна требовать участия нескольких лиц одновременно (т.е. должно быть достаточно одного человека).
6. От системы требуется, учитывая возможные обстоятельства её применения, чтобы она была проста в использовании, не требовала значительного умственного напряжения или соблюдения большого количества правил.

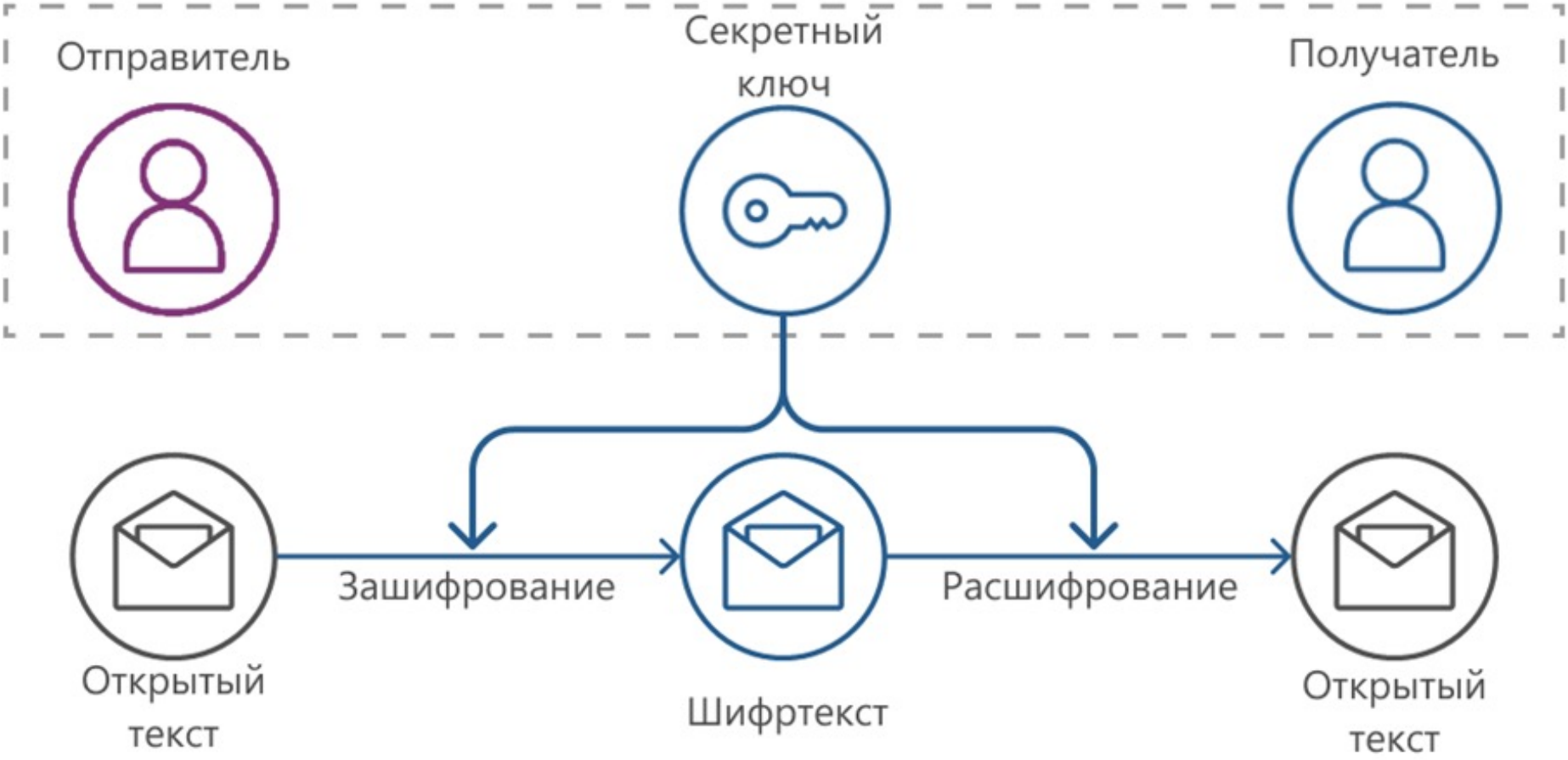
Основной принцип Керкгоффса: чем меньше секретов содержит система, тем выше её безопасность.

- Если утрата любого из секретов приводит к разрушению системы, то система с меньшим числом секретов будет надежнее.
- Чем больше секретов содержит система, тем более она ненадежна и потенциально уязвима.
- Чем меньше секретов в системе – тем выше ее прочность.

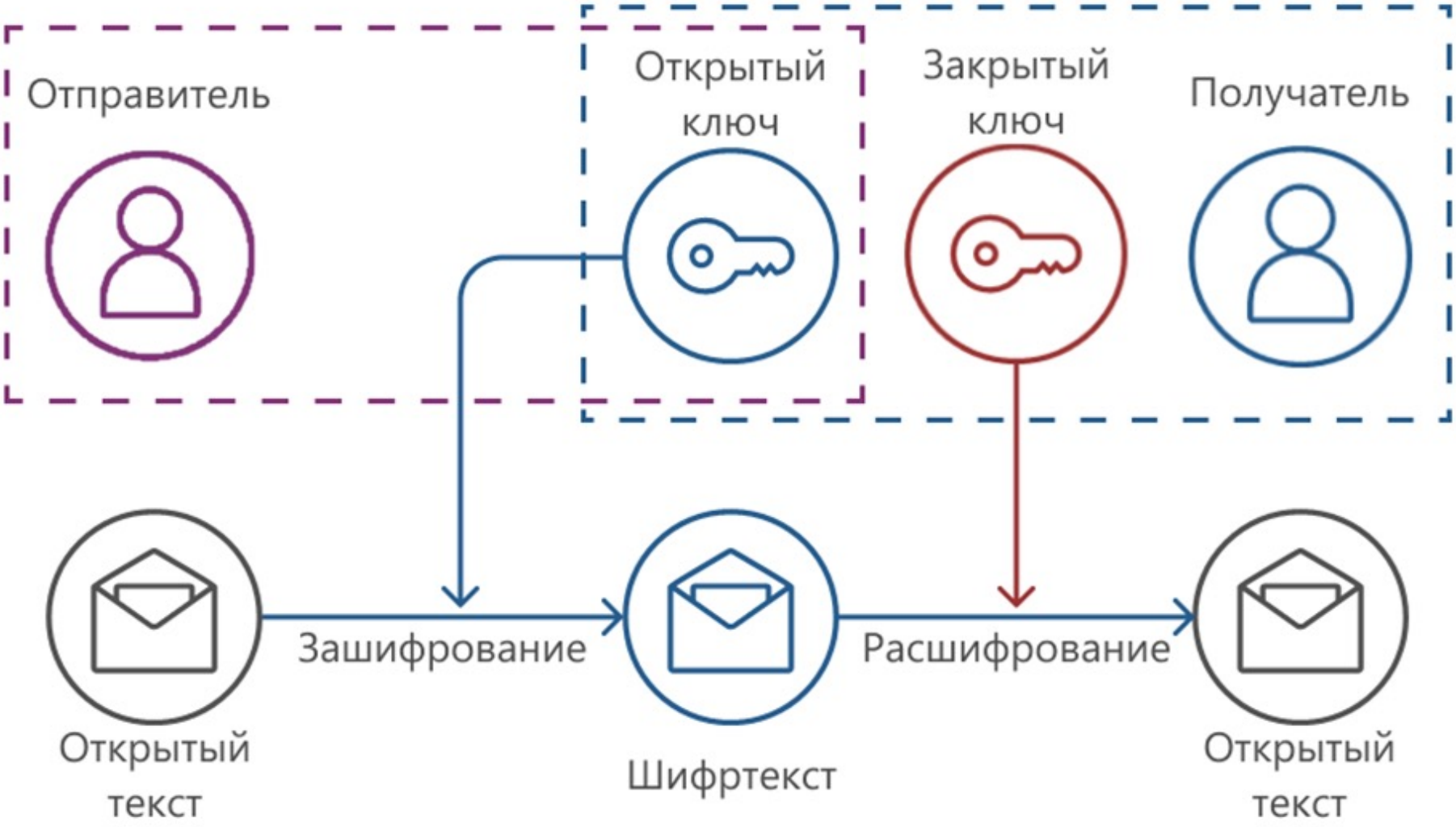
!!! Безопасность алгоритмов и протоколов должна быть независимой от их секретности; открытость не должна влиять на безопасность.

- Поэтому большинство систем шифрования используют известные, не составляющие секрета криптографические алгоритмы.
- Однако шифры, используемые **в правительственной и военной связи**, как правило, **засекречены**; таким образом создаётся «дополнительный рубеж обороны».

Симметричное шифрование



Асимметричное шифрование



Примеры задач, решаемых криптографией с открытым ключом:

- Хранение паролей в компьютере.
- Верификация в системах ПВО (при пересечении самолетом границы радиолокатор спрашивает пароль).
- Удаленный доступ через компьютерные сети (например, при взаимодействии банка и клиента). Обычно в начале сеанса банк запрашивает у клиента имя, а затем секретный пароль, но злоумышленник может узнать пароль, так как линия связи открытая.

Асимметричное шифрование в криптографии основано на **односторонних функциях** (one-way functions).

Зачастую используется функция возведения в степень по модулю:

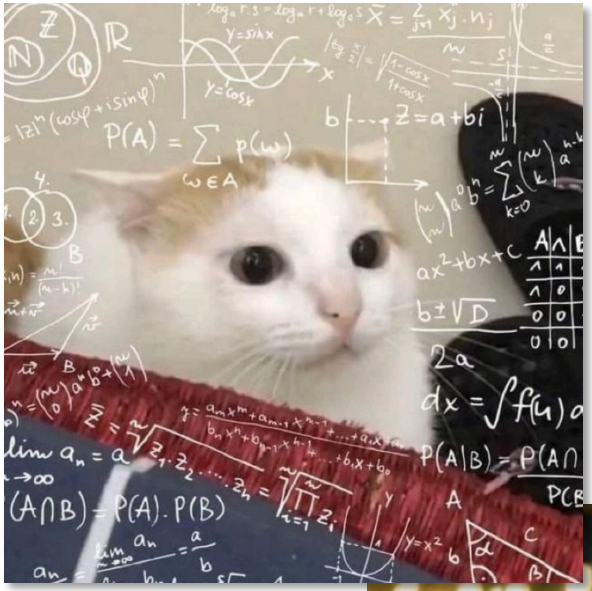
$$y = a^x \bmod p$$

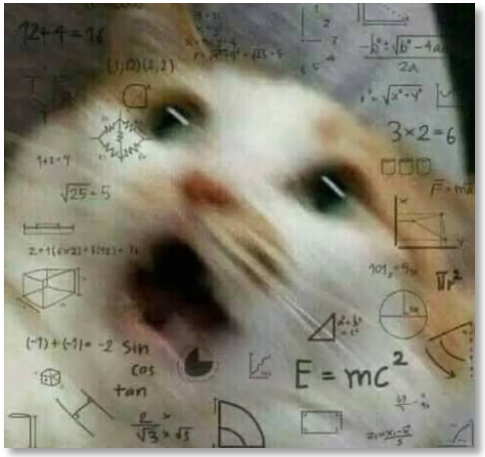
Обратная функция (дискретный логарифм) обозначается:

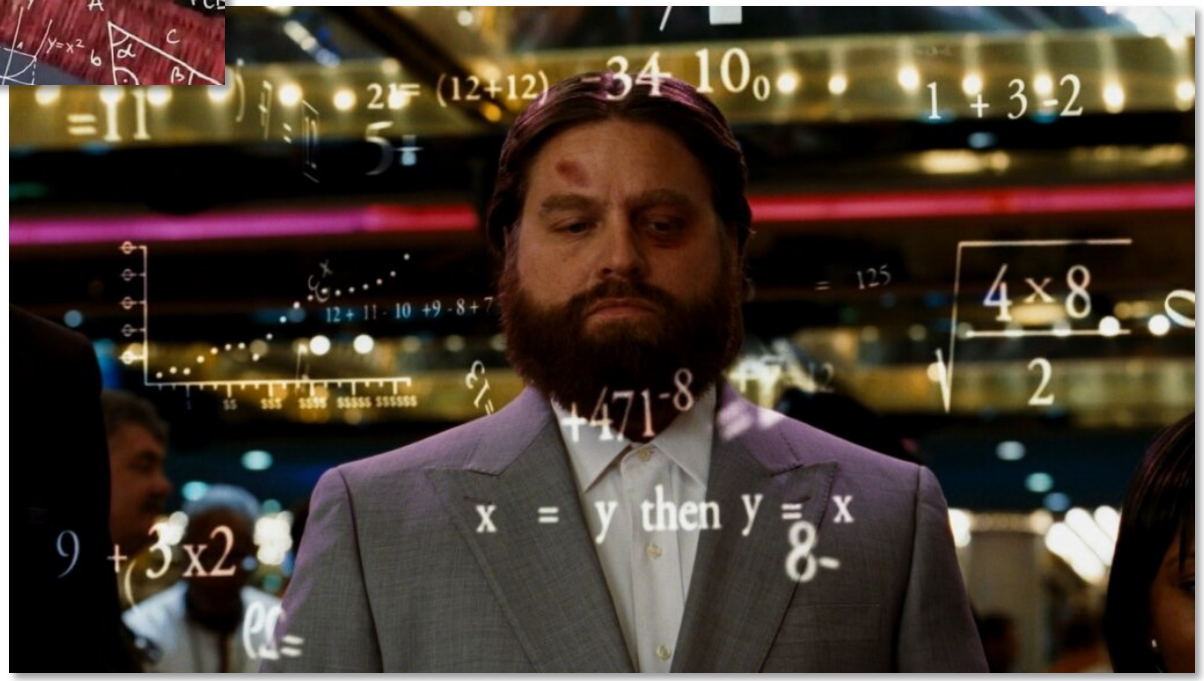
$$x = \log_a y \bmod p$$

При больших модулях (более 512 бит) такую обратную функцию (дискретный логарифм) практически невозможно вычислить.

Элементы теории чисел







Элементы теории чисел

Определение 2.2. Целое положительное число p называется *простым*, если оно не делится ни на какое другое число, кроме самого себя и единицы.

Пример 2.3. Числа 11, 23 — простые; числа 27, 33 — составные (27 делится на 3 и на 9, 33 делится на 3 и на 11).

Теорема 2.3 (основная теорема арифметики). *Любое целое положительное число может быть представлено в виде произведения простых чисел, причем единственным образом.*

Пример 2.4. $27 = 3 \cdot 3 \cdot 3,\ 33 = 3 \cdot 11.$

Определение 2.3. Два числа называются *взаимно простыми*, если они не имеют ни одного общего делителя кроме единицы.

Пример 2.5. Числа 27 и 28 взаимно просты (у них нет общих делителей кроме единицы), числа 27 и 33 — нет (у них есть общий делитель 3).

Определение 2.4 (функция Эйлера). Пусть дано целое число $N \geq 1$. Значение *функции Эйлера* $\varphi(N)$ равно количеству чисел в ряду $1, 2, 3, \ldots, N-1$, взаимно простых с N.

Пример 2.6.

$\varphi(10) = ?$
$1, \not{2}, 3, \not{4}, \not{5}, \not{6}, 7, \not{8}, 9,$
$\varphi(10) = 4$

$\varphi(12) = ?$
$1, \not{2}, \not{3}, \not{4}, 5, \not{6}, 7, \not{8}, \not{9}, \not{10}, 11$
$\varphi(12) = 4$

(здесь зачеркнуты числа, не взаимнопростые с аргументом).

Утверждение 2.4. *Если p — простое число, то $\varphi(p) = p - 1$.*

Доказательство. В ряду $1, 2, 3, \ldots, p-1$ все числа взаимно просты с p, так как p — простое число и по определению не делится ни на какое другое число.

Утверждение 2.5. *Пусть p и q — два различных простых числа ($p \neq q$). Тогда $\varphi(pq) = (p-1)(q-1)$.*

Доказательство. В ряду $1, 2, \ldots, pq-1$ не взаимнопростыми с pq будут числа

$$p, 2p, 3p, \cdots, (q-1)p$$

и

$$q, 2q, 3q, \cdots, (p-1)q.$$

Всего таких чисел будет $(q-1)+(p-1)$. Следвательно, количество чисел, взаимнопростых с pq, будет $pq-1-(p-1)-(q-1) = pq-q-p+1 = (p-1)(q-1)$.

Теорема 2.6 (Ферма). *Пусть p — простое число и $0 < a < p$. Тогда*

$$a^{p-1} \bmod p = 1.$$

Пример 2.7. $p = 13,\ a = 2;$

$$2^{12} \bmod 13 = \left(2^2\right)^2 \cdot \left(\left(2^2\right)^2\right)^2 \bmod 13 = 3 \cdot 9 \bmod 13 = 1,$$

$$10^{10} \bmod 11 = 10^2 \cdot \left(\left(10^2\right)^2\right)^2 \bmod 11 = 1 \cdot 1 = 1.$$

Теорема 2.7 (Эйлер). *Пусть a и b — взаимно простые числа. Тогда*

$$a^{\varphi(b)} \bmod b = 1.$$

Теорема Ферма является частным случаем теоремы Эйлера, когда b — простое число.

Пример 2.8.

$$\varphi(12) = 4,$$

$$5^4 \bmod 12 = \left(5^2\right)^2 \bmod 12 = \left(1^2\right)^2 \bmod 12 = 1.$$

$$\varphi(21) = 2 \cdot 6 = 12,$$

$$2^{12} \bmod 21 = 2^4 \cdot \left(2^4\right)^2 \bmod 21 = 16 \cdot 4 \bmod 21 = 1.$$

Нам понадобится еще одна теорема, близкая к теореме Эйлера.

Теорема 2.8. *Если p и q — простые числа, $p \neq q$ и k — произвольное целое число, то*

$$a^{k\varphi(pq)+1} \bmod (pq) = a. \tag{2.12}$$

Пример 2.9. Возьмем $p = 5$, $q = 7$. Тогда $pq = 35$, а функция Эйлера — $\varphi(35) = 4 \cdot 6 = 24$. Рассмотрим случай $k = 2$, т.е. будем возводить числа в степень $2 \cdot 24 + 1 = 49$. Получим

$$9^{49} \bmod 35 = 9, \quad 23^{49} \bmod 35 = 23.$$

Это не удивительно, так как каждое из чисел 9 и 23 взаимно просто с модулем 35, и по теореме Эйлера $9^{24} \bmod 35 = 1$, $23^{24} \bmod 35 = 1$. Однако теорема 2.8 остается верной и для следующих чисел:

$$10^{49} \bmod 35 = 10, \quad 28^{49} \bmod 35 = 28,$$

в то время как теорема Эйлера для них не применима (каждое из чисел 10 и 28 не взаимно просто с модулем 35 и $10^{24} \bmod 35 = 15$, $28^{24} \bmod 35 = 21$). □

Быстрое возведение в степень

В качестве примера односторонней функции рассмотрим следующую:

$$y = a^x \bmod p, \tag{2.3}$$

где p — некоторое простое число (т.е. такое, которое делится без остатка только на себя и на единицу), а x — целое число из множества $\{1, 2, \ldots, p-1\}$.

Обратная функция обозначается

$$x = \log_a y \bmod p \tag{2.4}$$

и называется *дискретным логарифмом*.

Сначала мы покажем, что вычисление по (2.3) может быть выполнено достаточно быстро. Начнем с примера вычисления числа $a^{16} \mathrm{mod} p$. Мы можем записать

$$a^{16} \mathrm{mod} p = \left(\left((a^2)^2 \right)^2 \right)^2 \mathrm{mod} p,$$

т.е.значение данной функции вычисляется всего за 4 операции умножения вместо 15 при «наивном» варианте $a \cdot a \cdot \ldots \cdot a$. На этом основан общий алгоритм.

Для описания алгоритма введем величину $t = \lfloor \log_2 x \rfloor$ — целую часть $\log_2 x$ (далее все логарифмы будут двоичные, поэтому в дальнейшем мы не будем указывать основание 2). Вычисляем числа ряда

$$a, \quad a^2, \quad a^4, \quad a^8, \quad \ldots, \quad a^{2^t} \qquad (\mathrm{mod}\ p). \tag{2.5}$$

$$ab \bmod n = ((a \bmod n)(b \bmod n)) \bmod n$$

В ряду (2.5) каждое число получается путем умножения предыдущего числа самого на себя по модулю p. Запишем показатель степени x в двоичной системе счисления:

$$x = (x_t x_{t-1} \ldots x_1 x_0)_2.$$

Тогда число $y = a^x \bmod p$ может быть вычислено как

$$y = \prod_{i=0}^{t} a^{x_i \cdot 2^i} \bmod p \qquad (2.6)$$

(все вычисления проводятся по модулю p).

Утверждение 2.1 (о сложности вычислений (2.3)). *Количество операций умножения при вычислении* (2.3) *по описанному методу не превосходит* $2\log x$.

Доказательство. Для вычисления чисел ряда (2.5) требуется t умножений, для вычисления y по (2.6) не более, чем t умножений (см. пример 2.1). Из условия $t = \lfloor \log x \rfloor$, учитывая, что $\lfloor \log x \rfloor \leq \log x$, делаем вывод о справедливости доказываемого утверждения. □

Замечание. Как будет показано в дальнейшем, при возведении в степень по модулю p имеет смысл использовать только показатели $x < p$. В этом случае мы можем сказать, что количество операций умножения при вычислении (2.3) не превосходит $2\log p$.

Важно отметить, что столь же эффективные алгоритмы вычисления обратной функции (2.4) неизвестны. Один из методов вычисления (2.4) называется «шаг младенца, шаг великана». Этот метод требует порядка $2\sqrt{p}$ операций.

Покажем, что при больших p функция (2.3) действительно односторонняя, если для вычисления обратной функции используется метод «шаг младенца, шаг великана. Получаем следующий результат (табл. 2.1).

Таблица 2.1. **Количество умножений для вычисления прямой и обратной функции**

Количество десятичных знаков в записи p	Вычисление (2.3) ($2\log p$ умножений)	Вычисление (2.4) ($2\sqrt{p}$ умножений)
12	$2 \cdot 40 = 80$	$2 \cdot 10^6$
60	$2 \cdot 200 = 400$	$2 \cdot 10^{30}$
90	$2 \cdot 300 = 600$	$2 \cdot 10^{45}$

Мы видим, что если использовать модули, состоящие из 50–100 десятичных цифр, то «прямая» функция вычисляется быстро, а обратная практически не вычислима. Рассмотрим, например, суперкомпьютер, который умножает два 90-значных числа за 10^{-14} сек. (для современных компьютеров это пока не доступно). Для вычисления (2.3) такому компьютеру потребуется

$$T_{\text{выч.пр.}} = 600 \cdot 10^{-14} = 6 \cdot 10^{-12} \text{ сек.},$$

а для вычисления (2.4) —

$$T_{\text{выч.обр.}} = 10^{45} \cdot 10^{-14} = 10^{31} \text{ сек.},$$

т.е. более 10^{22} лет.

Виды атак

- **По шифротексту.** Ева перехватила зашифрованный текст и пытается его дешифровать.
- **По известному тексту**. Ева получила какие-то исходные (открытые) тексты, соответствующие ранее переданным зашифрованным.

  Сопоставляя пары «текст-шифротекст», Ева пытается узнать секретный ключ, чтобы с его помощью дешифровать все последующие сообщения от Алисы к Бобу.
- **По выбранному тексту**. Противник пользуется не только предоставленными ему парами «текст-шифротекст», но может и сам формировать нужные ему тексты и шифровать их с помощью того ключа, который он хочет узнать.

Обобщенный алгоритм Евклида



Теорема 2.9. *Пусть a и b — два целых положительных числа. Тогда существуют целые (не обязательно положительные) числа x и y, такие, что*

$$ax + by = \gcd(a, b). \tag{2.13}$$

Обобщенный алгоритм Евклида служит для отыскания $\gcd(a, b)$ и x, y, удовлетворяющих (2.13). Введем три строки $U = (u_1, u_2, u_3)$, $V = (v_1, v_2, v_3)$ и $T = (t_1, t_2, t_3)$. Тогда алгоритм записывается следующим образом.

Алгоритм 2.2. Обобщенный алгоритм Евклида

ВХОД: Положительные целые числа a, b, $a \geq b$.
ВЫХОД: $\gcd(a,b)$, x, y, удовлетворяющие (2.13).

1. $U \leftarrow (a, 1, 0)$, $V \leftarrow (b, 0, 1)$.
2. WHILE $v_1 \neq 0$ DO
3. $q \leftarrow u_1 \text{ div } v_1$;
4. $T \leftarrow (u_1 \bmod v_1, u_2 - qv_2, u_3 - qv_3)$;
5. $U \leftarrow V$, $V \leftarrow T$.
6. RETURN $U = (\gcd(a,b), x, y)$.

Результат содержится в строке U.

Операция div в алгоритме — это целочисленное деление

$$a \text{ div } b = \lfloor a/b \rfloor.$$

Пример 2.12. Пусть a=28, b=19. Найдем числа x и y, удовлетворяющие (2.13).

U				28	1	0	
V	U			19	0	1	
T	V	U		9	1	-1	$q = 1$
	T	V	U	1	-2	3	$q = 2$
		T	V	0	19	-28	$q = 9$

Инверсия

Рассмотрим одно важное применение обобщенного алгоритма Евклида. Во многих задачах криптографии для заданных чисел c, m требуется находить такое число $d < m$, что

$$cd \bmod m = 1. \tag{2.14}$$

Отметим, что такое d существует тогда и только тогда, когда числа c и m взаимно простые.

Определение 2.6. Число d, удовлетворяющее (2.14), называется *инверсией c по модулю m* и часто обозначается $c^{-1} \bmod m$.

Данное обозначение для инверсии довольно естественно, так как мы можем теперь переписать (2.14) в виде

$$cc^{-1} \bmod m = 1.$$

Умножение на c^{-1} соответствует делению на c при вычислениях по модулю m. По аналогии можно ввести произвольные отрицательные степени при вычислениях по модулю m:

$$c^{-e} = (c^e)^{-1} = (c^{-1})^e \pmod{m}.$$

Пример 2.13. $3 \cdot 4 \bmod 11 = 1$, поэтому число 4 — это инверсия числа 3 по модулю 11. Можно записать $3^{-1} \bmod 11 = 4$. Число $5^{-2} \bmod 11$ может быть найдено двумя способами:

$$5^{-2} \bmod 11 = (5^2 \bmod 11)^{-1} \bmod 11 = 3^{-1} \bmod 11 = 4,$$

$$5^{-2} \bmod 11 = (5^{-1} \bmod 11)^2 \bmod 11 = 9^2 \bmod 11 = 4.$$

При вычислениях по второму способу мы использовали равенство $5^{-1} \bmod 11 = 9$. Действительно, $5 \cdot 9 \bmod 11 = 45 \bmod 11 = 1$.

Покажем, как можно вычислить инверсию с помощью обобщенного алгоритма Евклида. Равенство (2.14) означает, что для некоторого целого k

$$cd - km = 1. \tag{2.15}$$

Учитывая, что c и m взаимно просты, перепишем (2.15) в виде

$$m(-k) + cd = \gcd(m, c), \tag{2.16}$$

Пример 2.14. Вычислим $7^{-1} \bmod 11$. Используем такую же схему записи вычислений, как в примере 2.12 :

$$\begin{array}{rrl} 11 & 0 & \\ 7 & 1 & \\ 4 & -1 & q = 1 \\ 3 & 2 & q = 1 \\ 1 & -3 & q = 1 \\ 0 & 11 & q = 3. \end{array}$$

Получаем $d = -3$ и $d \bmod 11 = 11 - 3 = 8$, т.е. $7^{-1} \bmod 11 = 8$.
Проверим результат: $7 \cdot 8 \bmod 11 = 56 \bmod 11 = 1$.

Первая система с открытым ключом — система Диффи–Хеллмана

Эта криптосистема была открыта в середине 70-х годов американскими учеными:

- **Уитфилдом Диффи (Whitfield Diffie),**
- **Мартином Хеллманом (Martin Hellman)**.

Она привела к настоящей революции в криптографии и ее практических применениях.



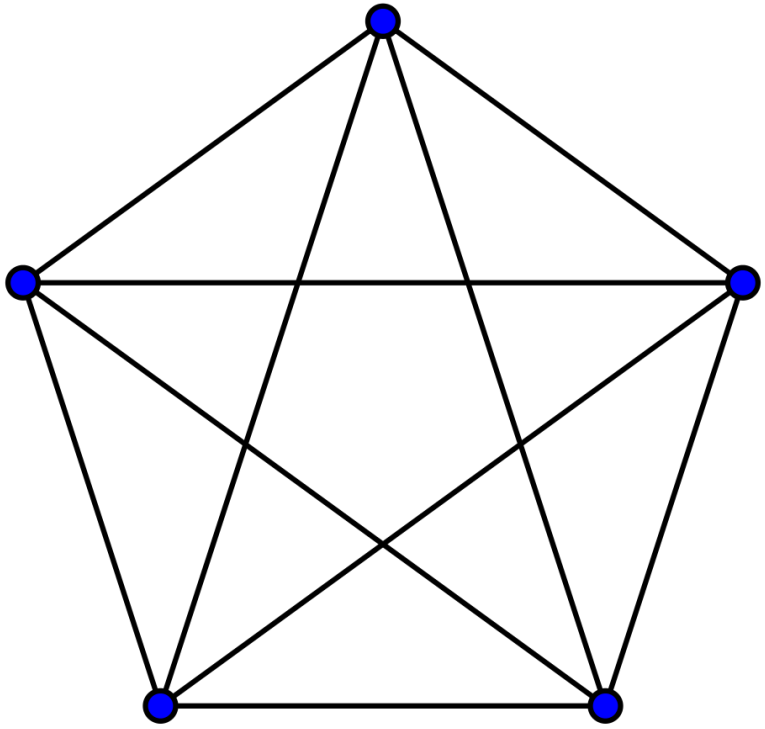
Это первая система, которая позволяла защищать информацию без использования секретных ключей, передаваемых по защищенным каналам.

На рисунке слева направо:

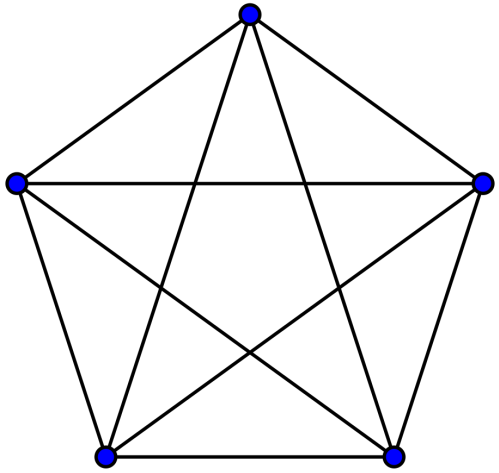
Мартин Хеллман, Уитфилд Диффи

Рассмотрим сеть связи с N пользователями, где N – большое число. Пусть мы хотим организовать секретную связь для каждой пары из них.

Тогда потребуется столько ключей: $C_N^2 = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$



Если мы будем использовать обычную систему распределения секретных ключей, то каждая пара абонентов должна быть снабжена своим секретным ключом, т.е. всего ключей:

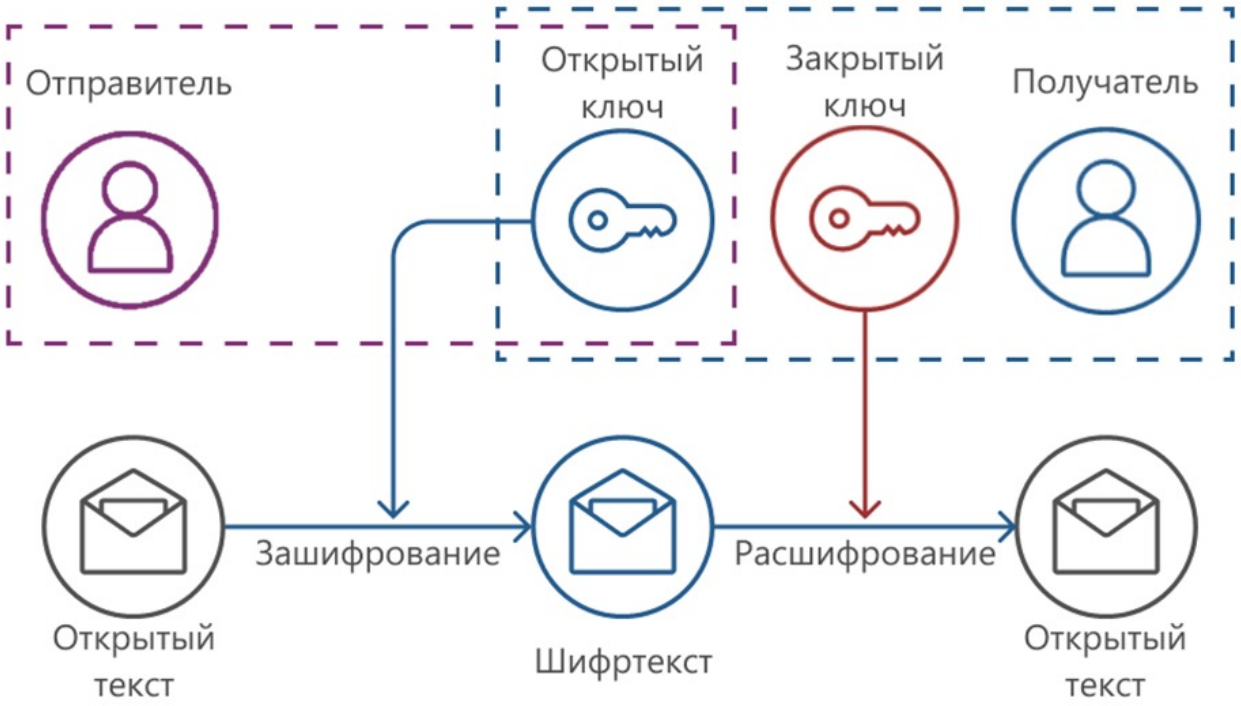


$$C_N^2 = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$$

При большом числе абонентов система снабжения их секретными ключами становится очень громоздкой и дорогостоящей.

У. Диффи и М. Хеллман решили эту проблему за счет открытого распространения и вычисления ключей.

Впервые появилось асимметричное шифрование!



У каждого абонента есть своя открытая и закрытая (секретная) информация.

Схема Диффи-Хеллмана

Строится система для нескольких абонентов: A, B, C.

Общие открытые параметры системы:

p – большое простое число,

g – первообразный корень по модулю p, $1 < g < p - 1$

Секретные параметры абонентов:

X_A, X_B, X_C – большие случайные числа в диапазоне $[2; p - 1]$

Открытые параметры абонентов:

$$Y_A = g^{X_A} \bmod p,$$

$$Y_B = g^{X_B} \bmod p,$$

$$Y_C = g^{X_C} \bmod p.$$

Таблица 2.2. **Ключи пользователей в системе Диффи–Хеллмана**

Абонент	Секретный ключ	Открытый ключ
A	X_A	Y_A
B	X_B	Y_B
C	X_C	Y_C

Допустим, абонент A решил организовать сеанс связи с B, при этом обоим абонентам доступна открытая информация из табл. 2.2. Абонент A сообщает B по открытому каналу, что он хочет передать ему сообщение. Затем абонент A вычисляет величину

$$Z_{AB} = (Y_B)^{X_A} \bmod p \tag{2.10}$$

(никто другой кроме A этого сделать не может, так как число X_A секретно). В свою очередь, абонент B вычисляет число

$$Z_{BA} = (Y_A)^{X_B} \bmod p. \tag{2.11}$$

Первообразный корень по модулю **m** — целое число **g** такое, что

$$g^{\varphi(m)} \equiv 1 \pmod{m}$$

и

$$g^{\ell} \not\equiv 1 \pmod{m} \text{ при } 1 \leq \ell < \varphi(m),$$

где $\varphi(m)$ — функция Эйлера.

Чтобы не проверять все ℓ от 1 до $\varphi(m)$, достаточно проверить три условия:

1. Является ли g числом взаимно простым с m, и если нет - то это не первообразный корень.
2. Так как $\varphi(m)$ - всегда чётное число для всех $m > 2$, то $\varphi(m)$ имеет как минимум один простой делитель - простое число 2, следовательно, для того, чтобы отсеять значительное количество не-корней, достаточно проверить $g^{\varphi(m)/2} \not\equiv 1 \pmod{m}$ для числа, подходящего на первообразный корень по модулю m.[1] Если результат +1 $\equiv$ m , то g - не корень, в ином случае результат -1 $\equiv$ m, когда g - это возможно первообразный корень.
3. Далее следует убедиться, что $g^{\ell} \not\equiv 1 \pmod{m}$ для всех $\ell = \frac{\varphi(m)}{p}$, где p - простой делитель числа $\varphi(m)$, полученный в результате его факторизации.

В практических реализациях для X_A и X_B используются числа порядка 10^{100} и p порядка 10^{300}. Число **g** не обязано быть большим и обычно имеет значение в пределах первого десятка.

Другими словами, **первообразный корень** — это образующий элемент мультипликативной группы кольца вычетов по модулю **m**.

Сравнение двух целых чисел по модулю натурального числа m — математическая операция, позволяющая ответить на вопрос о том, дают ли два выбранных целых числа при делении на **m** один и тот же остаток.

Любое целое число при делении на **m** дает один из **m** возможных остатков: число от 0 до **m – 1**.

Это значит, что все целые числа можно разделить на **m** групп, каждая из которых отвечает определённому остатку от деления на **m**.

Эти группы называются **классами вычетов по модулю m**, а содержащиеся в них целые числа — **вычетами** по модулю **m**.

Арифметические операции с остатками чисел по фиксированному модулю образуют **модульную арифметику**, которая широко применяется в **математике**, **информатике** и **криптографии**.

Первообразный корень по модулю **m** — целое число **g** такое, что

$$g^{\varphi(m)} \equiv 1 \pmod{m}$$

и

$$g^{\ell} \not\equiv 1 \pmod{m} \text{ при } 1 \leq \ell < \varphi(m),$$

где $\varphi(m)$ — функция Эйлера.

Число 3 является первообразным корнем по модулю 7. Чтобы в этом убедиться, достаточно каждое число от 1 до 6 представить как некоторую степень тройки по модулю 7:

$$\begin{aligned}
3^1 &\equiv 3 \pmod{7} \\
3^2 &\equiv 2 \pmod{7} \\
3^3 &\equiv 6 \pmod{7} \\
3^4 &\equiv 4 \pmod{7} \\
3^5 &\equiv 5 \pmod{7} \\
3^6 &\equiv 1 \pmod{7}
\end{aligned}$$

Задача быстрого нахождения чисел *p* и *g*

При произвольно заданном ***p*** она может оказаться трудной задачей, связанной с разложением на простые множители числа ***p-1***.

Для обеспечения высокой стойкости рассмотренной системы число ***p-1*** должно обязательно содержать большой простой множитель (в противном случае можно быстро вычислить дискретный логарифм с помощью алгоритма Полига–Хеллмана).

Поэтому часто рекомендуют использовать следующий подход. Простое число p выбирается таким, чтобы выполнялось равенство

$$p = 2q + 1,$$

где q — также простое число. Тогда в качестве g можно взять любое число, для которого справедливы неравенства

$$1 < g < p - 1 \quad \text{и} \quad g^q \bmod p \neq 1.$$

При работе алгоритма каждая сторона:

1. Генерирует случайное натуральное число — закрытый ключ

2. Совместно с удалённой стороной устанавливает открытые параметры **p** и **g** (обычно значения **p** и **g** генерируются на одной стороне и передаются другой), где **p** является случайным простым числом,

а **q = (p-1)/2** также должно быть случайным простым числом (для повышения безопасности).

g является первообразным корнем по модулю p (также является простым числом).

3. Вычисляет открытый ключ, используя преобразование над закрытым ключом

$$Y_A = g^{X_A} \bmod p$$

$$Y_B = g^{X_B} \bmod p$$

4. Обменивается открытыми ключами с удалённой стороной

5. Вычисляет общий секретный ключ **Z**, используя открытый ключ удаленной стороны B и свой закрытый ключ.

На стороне A:

$$Z_{AB} = Y_B{}^{X_A} \ mod \ p =$$

$$= (g^{X_B} \ mod \ p)^{X_A} \ mod \ p =$$

$$= (g^{X_B})^{X_A} \ mod \ p =$$

$$= g^{X_B X_A} \ mod \ p$$

На стороне B:

$$Z_{BA} = Y_A{}^{X_B} \ mod \ p =$$

$$= (g^{X_A} \ mod \ p)^{X_B} \ mod \ p =$$

$$= (g^{X_A})^{X_B} \ mod \ p =$$

$$= g^{X_A X_B} \ mod \ p$$

Утверждение 2.2. $Z_{AB} = Z_{BA}$.

Доказательство. Действительно,

$$Z_{AB} = (Y_B)^{X_A} \bmod p = (g^{X_B})^{X_A} \bmod p = \\ = g^{X_A X_B} \bmod p = (Y_A)^{X_B} \bmod p = Z_{BA}.$$

Отметим главные свойства системы:

1) абоненты A и B получили одно и то же число $Z = Z_{AB} = Z_{BA}$, которое не передавалось по открытой линии связи;

2) Ева не знает секретных чисел $X_A, X_B, \ldots$, поэтому не может вычислить число Z_{AB} (вообще говоря, она могла бы попытаться найти секретное число X_A по Y_A (см. (2.9)), однако при больших p это практически невозможно (требуются миллионы лет)).

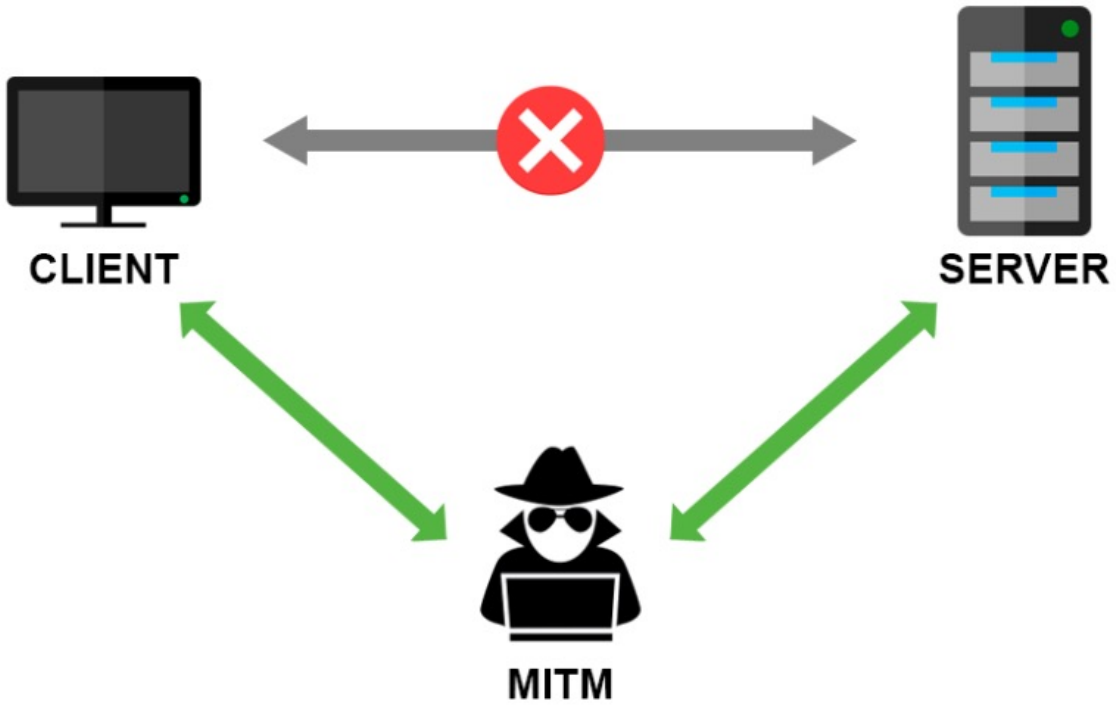
Пример 2.2. Пусть $p = 23 = 2 \cdot 11 + 1$ $(q = 11)$. Выберем параметр g. Попробуем взять $g = 3$. Проверим: $3^{11} \bmod 23 = 1$ и значит, такое g не подходит. Возьмем $g = 5$. Проверим: $5^{11} \bmod 23 = 22$. Итак, мы выбрали параметры $p = 23$, $g = 5$. Теперь каждый абонент выбирает секретное число и вычисляет соответствующее ему открытое число. Пусть выбраны $X_A = 7$, $X_B = 13$. Вычисляем

$Y_A = 5^7 \bmod 23 = 17$, $Y_B = 5^{13} \bmod 23 = 21$. Пусть A и B решили сформировать общий секретный ключ. Для этого A вычисляет $Z_{AB} = 21^7 \bmod 23 = 10$, а B вычисляет $Z_{BA} = 17^{13} \bmod 23 = 10$. Теперь они имеют общий ключ 10, который не передавался по каналу связи. □

Атака посредника - Man in the middle (MITM)

Злоумышленник тайно ретранслирует и при необходимости изменяет связь между двумя сторонами, которые считают, что они непосредственно общаются друг с другом.

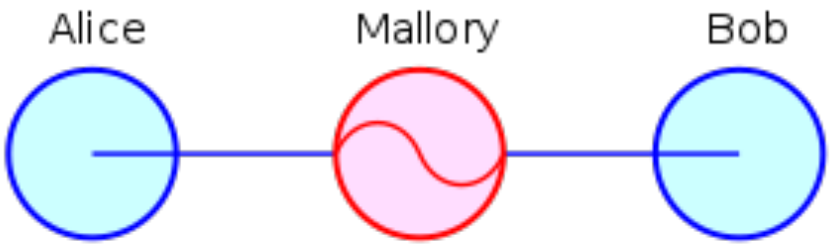
Взломщик осуществляет вмешательство в протокол передачи, **удаляя или искажая информацию**.



Данная атака направлена на обход взаимной аутентификации или отсутствие таковой.

Она будет успешной только тогда, когда злоумышленник имеет возможности:

- выдать себя за каждую конечную точку
- или оставаться незамеченным в качестве промежуточного узла.



Пример атаки на алгоритмическом языке

1. Алиса отправляет Бобу сообщение, которое перехватывает Мэлори:

   Алиса *«Привет, Боб, это Алиса. Пришли мне свой открытый ключ.»* → Мэлори
   Боб

2. Мэлори пересылает сообщение Бобу; Боб не может догадаться, что это сообщение не от Алисы:

   Алиса Мэлори *«Привет, Боб, это Алиса. Пришли мне свой открытый ключ.»* →
   Боб

3. Боб посылает свой ключ:

   Алиса Мэлори ← *[ключ Боба]* Боб

4. Мэлори подменяет ключ Боба своим и пересылает сообщение Алисе:

   Алиса ← *[ключ Мэлори]* Мэлори Боб

5. Алиса шифрует сообщение ключом Мэлори, считая, что это ключ Боба, и только он может расшифровать его:

   Алиса *«Встречаемся на автобусной остановке!» [зашифровано ключом Мэлори]* → Мэлори Боб

6. Мэлори расшифровывает сообщение, читает его, модифицирует его, шифрует ключом Боба и отправляет его:

   Алиса Мэлори *«Жди меня у входа в музей в 18:00.» [зашифровано ключом Боба]* → Боб

7. Боб считает, что это сообщение Алисы.

Атака на протокол Диффи-Хеллмана

Рассмотрим атаку на протокол выработки общего секрета Диффи-Хеллмана между сторонами ***A*** и ***B***. Допустим, криптоаналитик ***E*** имеет возможность не только перехватывать сообщения, но и подменять их своими, то есть осуществлять активную атаку:

$$A \leftrightsquigarrow E \leftrightsquigarrow B.$$

Перехват и подмена ключей

1. Сторона ***A*** отправляет сообщение стороне ***B***:

$$A \overset{E}{\rightsquigarrow} B: \ g^x \mod p.$$

2. Криптоаналитик ***E*** перехватывает сообщение стороны ***A*** и подменяет его, отправляя стороне ***B*** уже другое сообщение :

$$E \to B: \ g^z \mod p.$$

3. Сторона ***B*** отправляет сообщение стороне ***A***:

$$A \overset{E}{\leftsquigarrow} B: \ g^y \mod p.$$

4. Криптоаналитик ***E*** перехватывает сообщение стороны ***B*** и подменяет его, отправляя стороне ***A*** какое-то своё сообщение :

$$A \leftarrow E: \ g^z \mod p.$$

5. Результатом данных действий является образование двух каналов связи криптоаналитика ***E*** со сторонами ***A*** и ***B***, причем сторона ***A*** считает что общается со стороной ***B*** при помощи секретного ключа K_{AE}, а сторона ***B*** отправляет сообщения при помощи ключа K_{BE}. При этом стороны ***A*** и ***B*** не подозревают, что обмен сообщениями происходит не напрямую, а через криптоаналитика ***E***:

$$K_{AE} = g^{xz} \mod p,$$
$$K_{BE} = g^{yz} \mod p.$$

Подмена сообщений

1. Сторона ***A*** отправляет сообщение m стороне ***B***, зашифрованное при помощи ключа K_{AE}:

$$A \overset{E}{\nrightarrow} B : E_{K_{AE}}(m).$$

2. Криптоаналитик ***E*** перехватывает это сообщение, расшифровывает ключом K_{AE}, при необходимости, изменяет его на m', зашифровывает ключом K_{BE} и отправляет стороне ***B***: K_{AE}:

$$E \to B : E_{K_{BE}}(m').$$

3. Аналогичные действия криптоаналитик ***E*** предпринимает при передачи сообщений от ***B*** к ***A***.

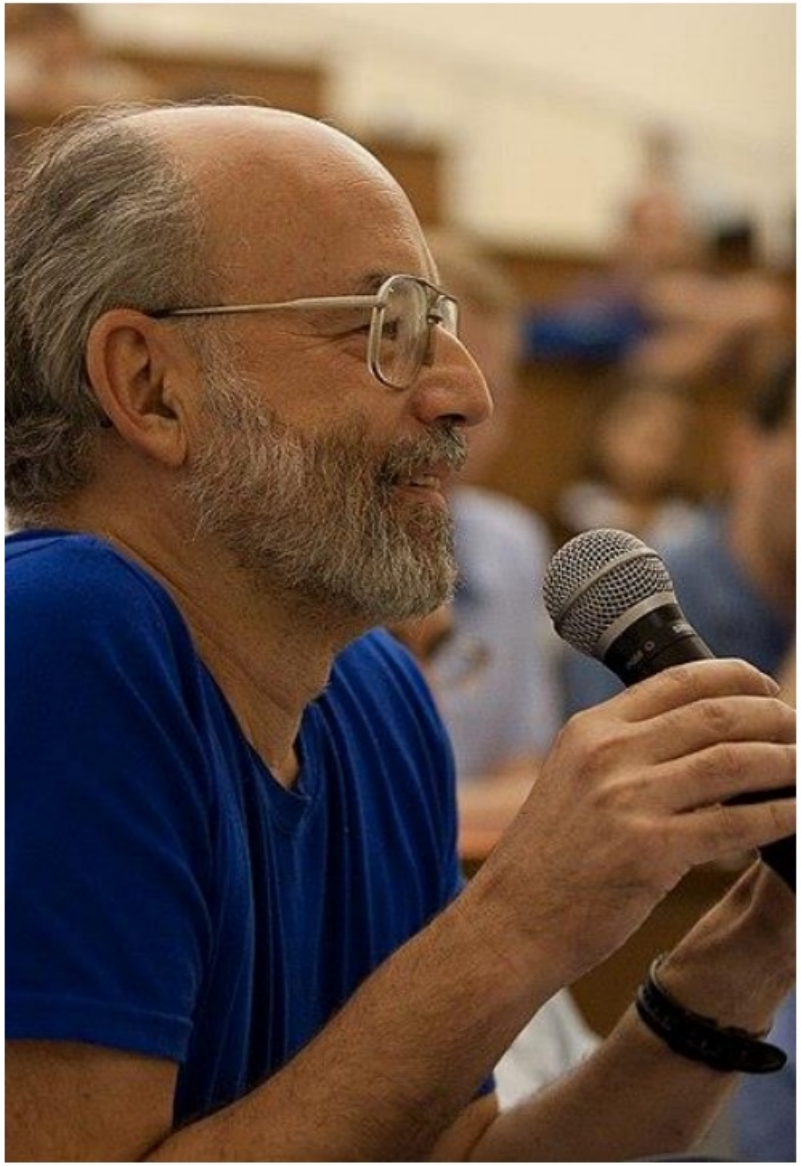
Сценарии атаки

- **Атака на системы с открытым ключом**
- **Внедрение вредоносного кода**
- **Downgrade Attack**
- **Публичные средства коммуникации**

Обнаружение MITM-атаки

- **IP-адрес сервера**
- **DNS-сервер**
- **X.509-сертификат сервера**
  - Подписан ли сертификат самостоятельно?
  - Подписан ли сертификат центром сертификации?
  - Был ли сертификат аннулирован?
  - Менялся ли сертификат недавно?
  - Получали ли другие клиенты в интернете такой же сертификат?

Шифр Шамира



~ 1980 г.

Ади Шамир (Adi Shamir)

описал трехэтапный протокол обмена зашифрованными сообщениями.

Описание алгоритма

Перейдем к описанию системы. Пусть есть два абонента A и B, соединенные линией связи. A хочет передать сообщение m абоненту B так, чтобы никто не узнал его содержание. A выбирает случайное большое простое число p и открыто передает его B. Затем A выбирает два числа c_A и d_A, такие, что

$$c_A d_A \bmod (p-1) = 1. \tag{2.17}$$

Эти числа A держит в секрете и передавать не будет. B тоже выбирает два числа c_B и d_B, такие, что

$$c_B d_B \bmod (p-1) = 1, \tag{2.18}$$

и держит их в секрете.

После этого A передает свое сообщение m, используя трехступенчатый протокол. Если $m < p$ (m рассматривается как число), то сообщение m передается сразу , если же $m \geq p$, то сообщение предствляется в виде $m_1, m_2, \ldots, m_t$, где все $m_i < p$, и затем передаются последовательно $m_1, m_2, \ldots, m_t$. При этом для кодирования каждого m_i лучше выбирать случайно новые пары (c_A, d_A) и (c_B, d_B) — в противном случае надежность системы понижается.

В настоящее время такой шифр, как правило, используется для передачи чисел, например, секретных ключей, значения которых меньше p. Таким образом, мы будем рассматривать только случай $m < p$. Дадим описание протокола.

Шаг 1. A вычисляет число

$$x_1 = m^{c_A} \bmod p, \tag{2.19}$$

где m — исходное сообщение, и пересылает x_1 к B.

Шаг 2. B, получив x_1, вычисляет число

$$x_2 = x_1^{c_B} \bmod p \tag{2.20}$$

и передает x_2 к A.

Шаг 3. A вычисляет число

$$x_3 = x_2^{d_A} \bmod p \tag{2.21}$$

и передает его B.

Шаг 4. B, получив x_3, вычисляет число

$$x_4 = x_3^{d_B} \bmod p. \tag{2.22}$$

Утверждение 2.10 (свойства протокола Шамира).

1) *$x_4 = m$, т.е. в результате реализации протокола от A к B действительно передается исходное сообщение;*

2) *злоумышленник не может узнать, какое сообщение было передано.*

Доказательство. Вначале заметим, что любое целое число $e \geq 0$ может быть представлено в виде $e = k(p-1) + r$, где $r = e \bmod (p-1)$. Поэтому на основании теоремы Ферма

$$x^e \bmod p = x^{k(p-1)+r} \bmod p = \\ = (1^k \cdot x^r) \bmod p = x^{e \bmod (p-1)} \bmod p. \quad (2.23)$$

Справедливость первого пункта утверждения вытекает из следующей цепочки равенств:

$$\begin{aligned} x_4 &= x_3^{d_B} \bmod p = (x_2^{d_A})^{d_B} \bmod p = \\ &= (x_1^{c_B})^{d_A d_B} \bmod p = (m^{c_A})^{c_B d_A d_B} \bmod p = \\ &= m^{c_A d_A c_B d_B} \bmod p = m^{(c_A d_A c_B d_B) \bmod (p-1)} \bmod p = m \end{aligned}$$

(предпоследнее равенство следует из (2.23), а последнее выполняется в силу (2.17) и (2.18)).

Доказательство второго пункта утверждения основано на предположении, что для злоумышленника, пытающегося определить m, не существует стратегии более эффективной, чем следующая. Вначале он вычисляет c_B из (2.20), затем находит d_B и, наконец, вычисляет $x_4 = m$ по (2.22). Но для осуществления этой стратегии злоумышленник должен решить задачу дискретного логарифмирования (2.20), что практически невозможно при больших p.

Опишем метод нахождения пар c_A, d_A и c_B, d_B, удовлетворяющих (2.17) и (2.18). Достаточно описать только действия для абонента A, так как действия для B совершенно аналогичны. Число c_A выбираем случайно так, чтобы оно было взаимно простым с $p-1$ (поиск целесообразно вести среди нечетных чисел, так как $p-1$ четно). Затем вычисляем d_A с помощью обобщенного алгоритма Евклида, как это было объяснено в разд. 2.3.

Пример 2.15. Пусть A хочет передать B сообщение $m = 10$. A выбирает $p = 23$, $c_A = 7$ $(\gcd(7, 22) = 1)$ и вычисляет $d_A = 19$. Аналогично, B выбирает параметры $c_B = 5$ (взаимно простое с 22) и $d_B = 9$. Переходим к протоколу Шамира.

Шаг 1. $x_1 = 10^7 \bmod 23 = 14$.

Шаг 2. $x_2 = 14^5 \bmod 23 = 15$.

Шаг 3. $x_3 = 15^{19} \bmod 23 = 19$.

Шаг 4. $x_4 = 19^9 \bmod 23 = 10$.

Таким образом, B получил передаваемое сообщение $m = 10$.

Методы взлома шифров, основанных на дискретном логарифмировании

Вычисления параметров во многих криптографических шифрах основываются на односторонних функциях.

$$y = a^x \bmod p$$

Обратная функция (дискретный логарифм) обозначается:

$$\mathrm{x} = \log_a y \bmod p$$

Можно вычислить ***y*** , если даны ***a*** и ***x***, затратив не более, чем **2 log *x*** операций.

Однако отыскание ***x*** по известным ***a*** и ***y*** , т.е. вычисление дискретного логарифма, — задача намного более трудная.

Достаточно рассматривать только показатели x, удовлетворяющие неравенству $0 \le x \le p-1$.

Обозначим через t_y число операций умножения, необходимых для вычисления y в (3.1) по a и x, и будем для краткости называть t_y временем вычисления. Время возведения в степень по алгоритмам из разд. 2.1 не больше $2\log x$, причем $x < p$. Отсюда

$$t_y \le 2\log p \qquad (3.2)$$

при любом показателе степени x.

Задача отыскания x в (3.1) по данным a и y.

Оценим сложность прямого перебора:

Сначала вычислить $\mathbf{a^1}$ и проверить, верно ли равенство $\mathbf{a^1 = y}$.
Если нет, то проверяем $\mathbf{a^2 = y}$, если нет, то $\mathbf{a^3 = y}$ и т.д. до $\mathbf{a^{p-1}}$.

В среднем потребуется **(p - 1) / 2** раз умножать на **a** и проверять равенство. Таким образом, время прямого перебора:

$$t_{\text{п.п.}} \approx p/2.$$

В методе «шаг младенца, шаг великана» время отыскания **x** существенно меньше:

$$t_{\text{ш.м.ш.в.}} \approx 2 \cdot \sqrt{p},$$

а в методе исчисления порядка это время еще меньше:

$$t_{\text{и.п.}} \approx c_1 \cdot 2^{c_2 \sqrt{\log p \log \log p}},$$

где c_1 , c_2 — некоторые положительные константы.

Чтобы сделать сравнение более наглядным, выразим время вычисления через длину числа p в (3.1). Обозначим эту длину в битах через n. При вычислениях по модулю p имеем $n \approx \log p$. Поэтому порядок трудоемкости (в смысле количества операций) упомянутых алгоритмов будет следующий:

$$\begin{aligned} t_y &\approx n, \\ t_{\text{п.п.}} &\approx 2^{n-1}, \\ t_{\text{ш.м.ш.в.}} &\approx 2^{n/2}, \\ t_{\text{и.п.}} &\approx 2^{c_2\sqrt{n \log n}}, \end{aligned}$$

где $\approx$ означает «пропорционально».

Метод «шаг младенца, шаг великана»

В открытой литературе этот метод был впервые описан **Дэниелом Шенксом** (Daniel Shanks) в 1973 году.

Это один из первых методов, который показал, что задача вычисления дискретного логарифма может быть решена значительно быстрее, чем методом перебора.

Описание метода «шаг младенца, шаг великана»:

Шаг 1. Сначала берем два целых числа m и k, такие, что

$$mk > p. \tag{3.3}$$

Шаг 2. Вычислим два ряда чисел

$$y, \quad ay, \quad a^2y, \quad \ldots, \quad a^{m-1}y \pmod{p}; \tag{3.4}$$

$$a^m, \quad a^{2m}, \quad \ldots, \quad a^{km} \pmod{p} \tag{3.5}$$

(все вычисления проводятся по модулю p).

Шаг 3. Найдем такие i и j, для которых выполняется равенство

$$a^{im} = a^j y. \tag{3.6}$$

Утверждение 3.1. *Число*

$$x = im - j \tag{3.7}$$

является решением уравнения (3.1). *Кроме того, целые числа* i, j, *удовлетворяющие* (3.6), *существуют.*

Доказательство. Справедливость (3.7) следует из приводимой ниже символической цепочки равенств, где все вычисления даны по модулю p, а деление соответствует умножению на обратный элемент:

$$a^x = a^{im-j} = \frac{a^{im}}{a^j} = \frac{a^{im}y}{a^j y} = \frac{a^{im}y}{a^{im}} = y.$$

Докажем теперь, что числа i и j, удовлетворяющие (3.6), существуют. Для этого сведем все числа вида (3.7) в таблицу 3.1.

Таблица 3.1. **Распределение чисел вида** $im - j$

$i \downarrow \quad j \rightarrow$	0	1	2	...	$m-1$
1	m	$m-1$	$m-2$	...	1
2	$2m$	$2m-1$	$2m-2$	...	$m+1$
...	...	...	...	...	...
k	km	$km-1$	$km-2$	...	$(k-1)m+1$

Мы видим, что в таблице содержатся все числа от 1 до km. Значит, из (3.3) следует, что в таблице содержатся все числа от 1 до p. Таким образом, любой показатель степени $x < p$ будет содержаться в таблице, т.е. число x, удовлетворяющее (3.1), может быть представлено в виде (3.7) и всегда найдется в таблице, поэтому уравнение (3.6) всегда имеет решение.

Пример 3.1. Найдем решение уравнения $2^x \bmod 23 = 9$, используя метод «шаг младенца, шаг великана».

Выберем m, k. Пусть $m = 6$, $k = 4$. Мы видим, что (3.3) выполняется. Вычислим числа (3.4), (3.5):

$$(3.4): \quad 9, 18, 13, 3, 6, 12;$$
$$(3.5): \quad 18.$$

Дальнейшие вычисления не проводим, так как уже нашлись одинаковые числа в (3.4) и (3.5) при $i = 1$, $j = 1$. По (3.7) получаем

$$x = 1 \cdot 6 - 1 = 5.$$

Проверим: $2^5 \bmod 23 = 9$. Действительно, $x = 5$ есть решение.

Имея два описанных ранее ряда, можно быстро найти в них равные элементы, воспользовавшись одним из описанных ниже методов.

Метод с использованием сортировки:

Пусть есть два массива $A[m], B[k]$ состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить в этих массивах равные элементы с трудоемкостью $O(m \log_2 m + k \log_2 k)$.

```
int A[m], B[k];
sort(A,A+m);
sort(B,B+k);
int i=0, j=0;
while(i<m && j<k)
{
  if(A[i]<B[j])
    ++i;
  else if(A[i]>B[j])
    ++j;
  else break;
}
if(i<m && j<k)
  cout << i << " " << j;
```

Метод с использованием структур данных:

Пусть есть два массива *A[m], B[k]* состоящие соответственно из m и k элементов. Ниже представлен код на языке C++, который позволяет находить равные элементы с трудоемкостью $O(m\log_2 m + k\log_2 k)$. Либо, в зависимости от используемой структуры данных, с трудоемкостью $O(mC + kC)$, где C - некоторая константа.

```
int A[m], B[k];
map<int,int> dict;
for(int i=0; i<m; ++i)
  dict[A[i]]=i;
for(int i=0; i<k; ++i)
  if(dict.count(B[i]))
  {
    cout << i << " " << dict[B[i]];
    break;
  }
```